

O PROCESSADOR MULTICICLO MIPS_S

1 CARACTERÍSTICAS GERAIS DA ARQUITETURA MIPS

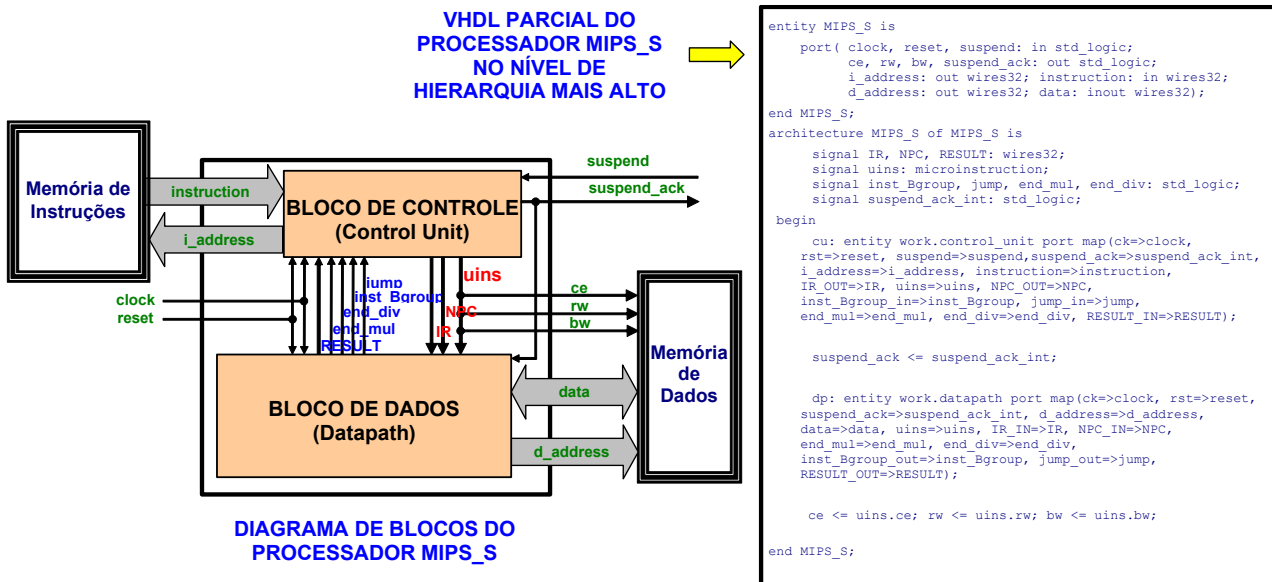
- A arquitetura MIPS é do tipo RISC *load-store*, ou seja, as instruções lógicas e aritméticas são executadas exclusivamente entre registradores da arquitetura ou entre constantes imediatas e registradores. As instruções de acesso à memória só executam apenas ou uma leitura da memória (*load*) ou uma escrita na memória (*store*). Todas as menções ao MIPS aqui se referem à primeira família original de arquiteturas MIPS-I, em particular ao primeiro componente desta, a R2000. Para mais detalhes sobre a variedade de processadores e famílias de processadores MIPS, vejam, por exemplo: https://en.wikipedia.org/wiki/MIPS_architecture.
- Para dar suporte à característica *load-store*, o processador disponibiliza um conjunto relativamente grande de registradores, para reduzir o número de acessos à memória externa, pois estes acessos representam perda de desempenho significativa em relação a operações entre registradores internos ao processador. Esta característica difere de arquiteturas baseadas em acumulador (e.g. o Motorola MC6800), que mantêm todos os dados em memória, realizando operações aritméticas entre um conteúdo que está em memória e um, ou uns poucos registradores de dados, o(s) denominado(s) acumulador(es). Considere o exemplo do código C a seguir: `for (i=0; i<1000; i++)`. Neste exemplo, caso `i` esteja armazenado em memória, durante a execução do laço tem-se 2.000 acessos à memória apenas ao valor de `i`, para realizar a leitura e a escrita deste a cada iteração do laço `for`. Caso se tenha o valor de `i` armazenado em um registrador interno, apenas opera-se sobre este, sem acesso à memória externa durante a maior parte do tempo. Considerando-se que o **tempo de acesso** a um registrador é normalmente de **uma a duas ordens de grandeza menor** que o tempo de acesso a uma posição de memória (ou seja, **10 a 100 vezes menor**, dependendo de aspectos tais como tecnologia de implementação do processador e das memórias, padrão de comunicação processador-memória etc.), percebe-se o ganho de desempenho que arquiteturas *load-store* podem auferir em relação a arquiteturas baseadas em acumulador(es).
- Dados e endereços na arquitetura MIPS são de 32 bits. Logo, diz-se que a **palavra** deste processador é de 32 bits, ou simplesmente que se trata de um processador de 32 bits. Obviamente, existem processadores com outro tamanho de palavra, tal como o I8051 da Intel (palavra de 8 bits), o TMS9900 da Texas (palavra de 16 bits) e o Cortex-A72 da ARM (64 bits).
- O endereçamento de memória no MIPS é orientado a byte, ou seja, cada endereço de memória é um identificador de uma posição de memória onde se armazena apenas 1 byte=8 bits. Então, uma palavra do processador armazenada em memória ocupa 4 posições de memória, tipicamente em quatro endereços consecutivos.
- O banco de registradores da arquitetura possui 32 registradores de uso geral, de 32 bits cada um, denominados **\$0** a **\$31**. O registrador **\$0** não é realmente um registrador, mas a constante 0 representada em 32 bits, que pode ser referenciada como um registrador ordinário. Este é disponibilizado para instruções que empregam o modo de endereçamento a registrador entre outros, e que necessitem usar o valor 0, muito útil na prática de programação em geral.
- Na arquitetura R2000 do MIPS há um tamanho regular definido para as instruções. Todas possuem exatamente o mesmo tamanho, e ocupam 1 palavra em memória (32 bits), ou seja, o equivalente a 4 endereços consecutivos em memória. A instrução contém o código da operação e o(s) operando(s), caso este(s) exista(m). Acesso a instruções são sempre alinhados em uma fronteira de palavra inteira. Assim, todo endereço válido de uma instrução possui os dois últimos bits iguais a 0 (e.g. endereços 0, 4, 16, 32008 etc. que podem ser representados em decimal ou em hexadecimal).
- Os quatro modos de endereçamento mais importantes usados nas instruções desta arquitetura são os modos *a registrador*, *imediato*, *base-deslocamento* e *relativo ao PC*. Há outros.
- O processador não possui suporte em hardware para gerenciar estruturas do tipo pilha em memória, mas existe um registrador do banco, o **\$sp** ou **\$29**, que associado a instruções operando sobre este, em particular com três instruções específicas (**JAL**, **JALR** e **JR**), permite simular estruturas do tipo pilha em memória para, por exemplo, realizar o salvamento de contexto de funções e sub-rotinas. Mas a diferenciação do registrador **\$sp** é meramente uma convenção (exceto pelo seu uso implícito na instrução **JAL**).
- Não existem registradores específicos para armazenar explicitamente qualificadores sobre a execução de instruções no MIPS. Exemplos destes são as informações de que a execução de uma instrução resultou na constante 0, ou que uma instrução aritmética gerou vai-um (em inglês, *carry*), ou ainda que seu resultado não é correto porque necessita recursos de armazenamento de maior capacidade que os disponíveis no processador

(situação chamada de transbordo, ou, em inglês, *overflow*). A detecção destas situações deve ser realizada usando instruções específicas de comparação disponíveis na arquitetura justamente para isto, que escrevem um resultado 0 ou 1 em um registrador de 32 bits (vejam o comportamento das instruções do MIPS cujo mnemônico possui o prefixo **SLT**).

O que é descrito neste documento é um processador que implementa um subconjunto da arquitetura do MIPS R2000 (donde o nome **MIPS_S**, abreviatura de **MIPS Subset** ou **Subconjunto do MIPS**). Trata-se praticamente de uma máquina RISC completa. Faltam, contudo algumas características que existem na maioria das organizações RISC práticas, e.g.: (1) organização *pipeline*; (2) estruturas mais sofisticadas para operações de entrada e saída tal controladores de interrupção e acesso direto à memória (do inglês *direct memory access*, ou DMA); (3) organizações de memória mais eficientes tais como as que empregam *caches*. Estes assuntos são deixados para serem introduzidos e estudados em maior profundidade em outra oportunidade. O objetivo da MIPS_S é dar apoio ao aprendizado mais fundamental de arquiteturas e organizações de processadores RISC.

2 A RELAÇÃO PROCESSADOR - MEMÓRIAS - AMBIENTE EXTERNO

A Figura 1(a) mostra a visão de alto nível de uma **proposta de organização** para implementar o processador MIPS_S. Nela, está explicitada a relação entre processador, suas memórias externas (de Instruções e Dados) e o mundo exterior ao subsistema processador. O mundo externo é o responsável por gerar os sinais de **clock** e **reset**. Também cabe ao mundo externo ao processador gerar eventualmente o sinal **suspend** (ver mais adiante). Uma característica primordial deste hardware é que se trata de uma **organização Harvard**, ou seja, o processador usa interfaces físicas distintas para comunicação com as memórias de instruções e de dados. Outra possibilidade seria o uso de uma interface de memória unificada para instruções e dados, o que caracterizaria uma **organização von Neumann**. Um detalhe importante é que embora sejam usadas duas memórias separadas, ambas compartilham o mesmo mapa de memória unificado da arquitetura MIPS. A separação entre as memórias se dá pelas regiões do mapa que as duas ocupam. Em consonância com a operação do simulador MARS, usado para desenvolver programas na linguagem de montagem do MIPS, a Memória de Instruções possui sua primeira posição associada ao endereço **0x00400000**¹ do mapa de memória do MIPS. Por outro lado, a Memória de Dados possui sua primeira posição associada ao endereço **0x10010000** do mapa de memória do MIPS. O tamanho de cada uma das duas memórias determina qual a última posição do mapa que realmente existe (fisicamente).



(a) Diagrama de blocos processador-memória para a organização MIPS_S.

(b) VHDL parcial do nível mais alto da hierarquia de descrição do processador MIPS_S.

Figura 1 – Proposta de uma organização para o processador MIPS_S. Ilustra-se: (a) a conectividade entre o processador, ambiente externo, memórias externas e (b) descrição VHDL parcial do módulo de nível hierárquico mais alto para a organização MIPS_S.

¹ A notação usada aqui, **0x00400000** é uma forma de representar números na base hexadecimal. Adota-se a convenção usada no montador/simulador MARS do MIPS, que emprega o prefixo “0x” para indicar que um número que o segue está representado na base 16.

Observação Importante: A organização aqui descrita do MIPS_S é adequada sobretudo para simulação. Para prototipar a MIPS_S, algumas alterações na interface e na operação interna facilitam o uso didático da arquitetura. Desta forma, a organização particular descrita aqui é a MIPS_S_Sim (o sufixo _Sim indicando a versão simulável). Existe uma segunda organização, levemente diferente, a MIPS_S_Prt (o sufixo _Prt indicando a versão prototipável). Esta última será descrita e diferenciada da MIPS_S_Sim oportunamente. No restante deste documento, referências à organização MIPS_S_Sim usam o termo genérico MIPS_S.

O tamanho das memórias não é definido neste documento e claro, pode variar, principalmente porque as memórias não são parte do processador MIPS_S. Assim, por exemplo, caso se deseje empregar estruturas de pilha, é preciso determinar um valor inicial adequado para o registrador \$sp. De fato, este valor não será o valor inicial usado por omissão no MARS (\$sp=0x7FFFFFFF), pois o tamanho da Memória de Dados teria de ser muito grande para ser prático usá-lo. Uma sugestão é inicializar em software o \$sp com o primeiro endereço após o fim da memória de dados, pois a pilha cresce para endereços menores. No *testbench* do exemplo usado para simular o MIPS_S, este valor é 0x10012000, pois a memória de Instruções tem tamanho de 2Kbytes e a memória de Dados tem tamanho de 8Kbytes. Este tamanho também é assumido na descrição do processador em si, ao criar o \$sp.

O processador MIPS_S recebe do mundo externo três sinais de controle. O primeiro, denominado **clock**, sincroniza todos os eventos internos do processador. O sinal **reset** leva o processador a reiniciar a execução de instruções a partir do endereço 0x00400000 de memória (este endereço é definido por compatibilidade com o endereço assumido pelo simulador MARS como aquele de início da área de programas). O sinal **reset** pode ser usado para provocar o reinício da operação do processador. O terceiro sinal, **suspend**, quando ativado, faz com que o MIPS_S pause o processo de execução de instruções, após concluir de executar a instrução corrente quando da ativação deste sinal.

Os sinais providos pelo processador MIPS_S para a troca de informações com o mundo externo a ele, representado pelas memórias e pelos sinais **clock**, **reset**, **suspend** e **suspend_ack** são:

- ♦ **i_address** – um barramento unidirecional de 32 bits: define sempre o endereço da posição da Memória de Instruções contendo a instrução a ser buscada a seguir;
- ♦ **instruction** – um barramento unidirecional de 32 bits: contém sempre o código objeto da instrução contida na posição da Memória de Instruções apontada por **i_address**;
- ♦ **d_address** – um barramento unidirecional de 32 bits: contém o endereço da posição da Memória de Dados a ser lida ou escrita, respectivamente;
- ♦ **data** - um barramento bidirecional de 32 bits: transporta dados do ou para o processador MIPS_S para ou da Memória de Dados;
- ♦ **ce, rw, bw** – os sinais de *status* que controlam a operação da Memória de Dados. Se **ce**=‘1’ uma operação com a memória de dados está em curso. Neste caso, **rw**=‘1’ indica uma leitura da memória e **rw**=‘0’ indica uma escrita na memória. Leituras sempre trazem 32 bits da memória. A escrita na memória, por outro lado, pode ser de 4 bytes ou de apenas 1 byte. Quando se realiza uma escrita com **bw**=‘0’, 4 bytes são escritos na memória. Quando **bw**=‘1’, apenas 1 byte é escrito na memória, aquele que está no byte menos significativo do barramento **data**.
- ♦ **suspend, suspend_ack** – o sinal de entrada de controle **suspend** ativo em ‘1’, pode ser usado pelo mundo externo para temporariamente pausar a execução de instruções pelo MIPS_S. O mundo externo ativa este sinal e aguarda que o MIPS_S coloque sua saída de *status* **suspend_ack** em ‘1’, o que indica que o processador está suspenso e liberou o acesso à interface da memória de dados (sinais **d_address**, **data**, **ce**, **rw** e **bw**) para o mundo externo. Note-se que o processador sempre conclui a execução da instrução que está sendo realizada no momento antes de pausar. Isto faz com que a ativação de **suspend_ack** dependa deste processo, que pode assim demorar entre 1 e 67 ciclos de relógio para ocorrer². O sinal **suspend_ack** é registrado na borda de subida do relógio, para facilitar a interação e sincronização entre o MIPS_S e o mundo externo.

Além dos sinais que permitem a interação com o mundo externo, os principais componentes da organização do processador MIPS_S, o Bloco de Controle (*control_unit*, ou BC) e o Bloco de Dados (*datapath*, ou BD) interagem através de um conjunto de sinais específicos. São 4 sinais que passam do BC para o BD e 5 sinais qualificadores que passam do BD para o BC.

Os sinais gerados no BC e enviados ao BD são:

- ❖ A palavra de microinstrução (**uins**), para comandar a execução passo a passo das instruções pelo BD. A microinstrução é responsável por especificar cada uma das ações unitárias que serão executadas pelo hardware do BD a cada ciclo de relógio, também chamadas de *micro-operações*. Exemplos destas são cada uma das três

² Este último valor corresponde ao número de ciclos para executar a instrução mais demorada da MIPS_S. Ver Tabela 4.

seleções de registradores a serem escritos/lidos no/do banco de registradores, a operação que a Unidade Lógico-aritmética (ULA ou, usando a terminologia em inglês, *Arithmetic Logic Unit* ou *ALU*) executará, e os sinais de controle de acesso à Memória de Dados;

- ❖ O endereço potencial da próxima instrução a ser executada (conteúdo do registrador **NPC**, localizado no BC);
- ❖ O código objeto da instrução corrente (conteúdo do registrador **IR**, localizado no BC);
- ❖ O sinal **suspend_ack**, indicando ao BD que o processador está em suspensão.

Os sinais gerados no BD e enviados ao BC são:

- ❖ O sinal **jump**, que contém o resultado do teste de condições de salto nas instruções de salto condicional;
- ❖ O sinal **inst_Bgroup**, que qualifica se a instrução ora em execução é (ou não) uma das de salto condicional;
- ❖ Os qualificadores **end_mul** e **end_div**, produzidos no BD pelos operadores de multiplicação e divisão, respectivamente. Estes informam ao BC o final da execução destas operações com um pulso de duração de 1 ciclo de relógio em '1'. Isto é necessário porque se trata de operações mais complexas que gastam, nesta organização, dezenas de ciclos de relógio para executar. O BC comanda o início da execução delas e aguarda pela informação de seu término para seguir com o fluxo normal de ciclos de controle;
- ❖ O sinal **RESULT**, que eventualmente contém um endereço calculado durante instruções de salto (incondicionais ou condicionais), valor este possivelmente escrito no registrador **PC**, que se encontra no BC.

É importante ressaltar que o BC e o BD operam sempre e apenas na borda de subida do sinal de relógio (**clock**). A cada borda de subida do **clock** (e após a desativação do sinal de **reset**), o BC gera uma palavra de microinstrução, e na borda de subida seguinte o BD executa as ações comandadas por esta, modificando seus registradores. É necessário que sempre existam dados estáveis antes das bordas de subida relógio em cada um dos blocos³.

A Figura 1(b) representa um exemplo de codificação textual do nível mais alto da hierarquia do processador, na linguagem VHDL. Neste texto deve-se notar que o BC e o BD são instanciados e conectados entre si por *sinais*, conforme definido pelos comandos *port map* no processo de instanciação.

2.1 Estrutura de Acesso à Memória

A partir da discussão da Figura 1, percebe-se que ambas as interfaces de memória são assíncronas, ou seja, não dependem de sinais de relógio tais como o sinal **clock**. Cada uma das interfaces de acesso define um **mapa de memória**, conforme ilustrado na Figura 2. Um mapa de memória é uma abstração que permite organizar as informações às quais o processador tem acesso na forma de uma tabela. Cada posição deste mapa possui um endereço associado e armazena um valor. Na arquitetura MIPS_S, ambos os mapas (de instruções e de dados) possuem endereços de 32 bits e conteúdo de 8 bits associados a cada posição da tabela. Assim, diz-se que as memórias de instruções e de dados são **endereçadas a byte** (1 byte = 8 bits) e cada um destes mapas possui 2^{32} posições (4.294.967.296 posições, valor normalmente denominado 4Gigabytes ou 4GB, correspondendo ao número de endereços distintos de 32 bits, entre **0x00000000** e **0xFFFFFFFF**).

A Memória de Instruções armazena apenas instruções. Cada instrução na MIPS_S ocupa exatamente 32 bits. Assim, uma instrução ocupa 4 posições de memória. Como visto na Figura 1, contudo, o barramento de instruções é de 32 bits. Logo, uma leitura da memória de instruções busca desta não apenas o byte cujo endereço está no barramento **i_address**, mas este e os três seguintes (esta é uma regra ditada pela arquitetura). Uma outra limitação imposta pela arquitetura para facilitar o acesso à memória de instruções é alinhar instruções em uma fronteira inteira de palavra. Esta expressão significa que o código objeto de cada instrução da MIPS_S só pode começar a partir de um endereço múltiplo de 4, endereços cujos 2 últimos bits são 00. Assim, por exemplo, uma única instrução pode ocupar os endereços de 0xAAA00004 a 0xAAA00007 na memória de instruções, mas nunca os endereços 0xFF004677 a 0xFF00467A da mesma memória, embora ambas as faixas correspondam a 4 bytes consecutivos na memória. Dada esta situação, o barramento **i_address** não precisaria ter 32 bits, mas apenas 30, pois os 2 últimos bits deste endereço valem sempre "00". Não existem sinais de controle para acesso à Memória de Instruções além do endereço em **i_address**. Isto não é necessário, pois não há fluxo bidirecional de informação entre o processador e esta memória. A memória de instruções é vista pela organização MIPS_S como uma memória de apenas leitura, que fornece informações na sua saída (instruções) a partir do estabelecimento do endereço de memória pelo processador, via o barramento **i_address**.

³ Deve estar claro que esta última afirmação se verifica apenas se a frequência de relógio for suficientemente baixa para permitir que os sinais estabilizem em no máximo um período de relógio (entre duas bordas de subida consecutivas do **clock**).

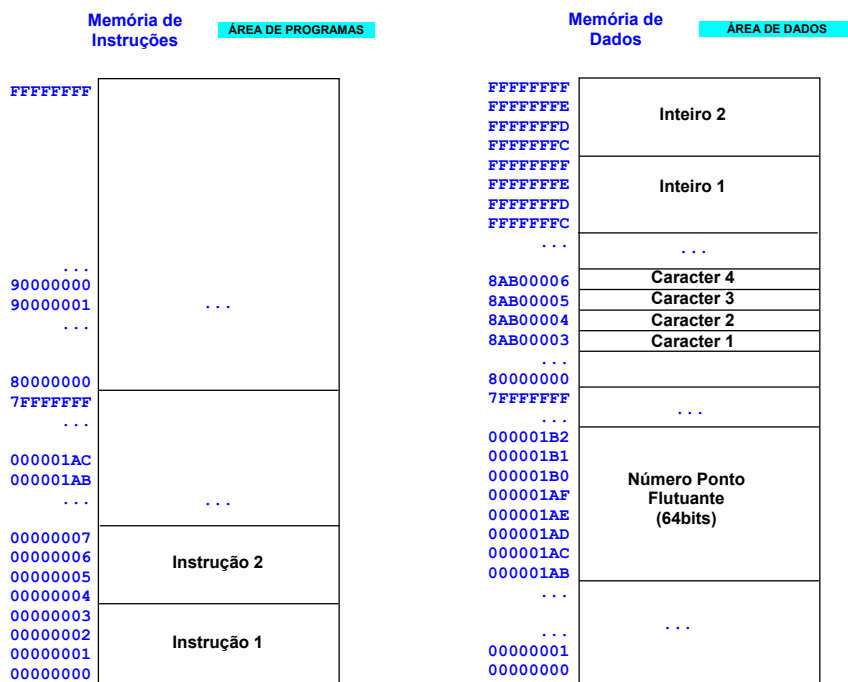


Figura 2 - Ilustração da estrutura dos mapas de memória da MIPS_S. Cada mapa possui 4Gposições de 1 byte. Cada instrução ocupa exatamente 4 bytes na memória de instruções, sempre alinhado a palavra.

A Memória de Dados possui uma interface um pouco mais elaborada, devido ao fato de poder ser lida ou escrita. Além disto, os dados lidos ou gravados nesta memória podem ter tamanho diversos. Estas necessidades fazem com que o acesso à informação deva ter um formato mais flexível. Por exemplo, um inteiro típico ocupa 32 bits, mas um caractere ocupa apenas 1 byte, e um número em ponto flutuante pode ser de 32 ou de 64 bits. Além disto, estruturas complexas, tais como aquelas usadas em programação de alto nível (e.g. struct de C/Java ou record de Pascal, e listas em LISP), podem ter tamanho variável, em alguns casos até sem limitações de tamanho mínimo ou máximo. Vetores e arranjos multidimensionais de números ou caracteres e de estruturas complexas complicam ainda mais a situação. Logo, a interface de acesso à Memória de Dados deve ser flexível para prover desempenho e proporcionar economia de espaço. Ainda assim, o barramento de dados (**data**) é fixo, de 32 bits. Lê-se sempre 4 bytes (ou posições) de memória de cada vez. O que é lido pode ser, por exemplo, 1 inteiro ou 4 caracteres ou metade de um número em ponto flutuante de precisão dupla. Como a leitura é alinhada em qualquer fronteira, é até mesmo possível ler metade de um inteiro e dois caracteres nos 32 bits.

O controle de acesso à memória de dados é feito pelo processador através dos sinais **ce**, **rw** e **bw**. O sinal **ce** indica se está em curso uma operação com a memória de dados (quando **ce**='1') e o sinal **rw** indica se esta operação é de escrita (quando **rw**='0') ou de leitura (quando **rw**='1'). Obviamente, quando **ce**='0' o valor do sinal **rw** é irrelevante. O sinal **bw** serve para indicar se uma operação de escrita na memória deve escrever apenas um byte na posição da memória de dados cujo endereço aparece no barramento **d_address** (**bw**='1') ou se esta deve escrever 4 bytes (ou seja, na posição explicitada na instrução e nas três seguintes), o que ocorre quando, em uma escrita, **bw**='0'.

2.2 Uma palavra sobre “Endianismo”

Processadores em geral podem ordenar os bytes dentro de uma palavra, meia palavra ou palavra dupla, fazendo o byte com menor índice ser aquele mais à esquerda ou aquele mais à direita. A convenção usada por uma máquina é a ordem escolhida para tanto. Processadores que usam a arquitetura MIPS podem operar com qualquer destas ordens. A ordem escolhida é denominada o “endianismo” do processador (mais precisamente, da organização deste processador) e pode ser “*big-endian*” ou “*little-endian*”. Processadores *big-endian* assumem os 8 bits mais à esquerda de uma palavra (para números a parte mais significativa deste) como sendo o byte 0, os 8 bits imediatamente à direita deste como o byte 1, etc. Processadores *little-endian* assumem os 8 bits mais à direita de uma palavra (para números a parte menos significativa deste) como sendo o byte 0, os 8 bits imediatamente à esquerda deste como o byte 1, etc. O simulador a ser usado (MARS) opera com ambas as ordens. A ordem específica do MARS é determinada pela ordem dos bytes da máquina subjacente que executa o simulador. Assim, ao rodar o simulador em uma máquina baseada em processador que use a arquitetura x86 da Intel, MARS é *little-endian* (pois a arquitetura Intel x86 é *little-endian*), enquanto em alguns servidores baseados em processadores SPARC da Oracle/Sun ou da Fujitsu MARS é *big-endian* (pois a arquitetura SPARC é *big-endian*).

3 O CONJUNTO DE INSTRUÇÕES DO MIPS_S

A Tabela 1 resume o conjunto de instruções da arquitetura MIPS_S. Note-se que todas as 37 instruções listadas aqui existem exatamente na mesma forma na arquitetura MIPS, justificando dizer que a arquitetura MIPS_S é um subconjunto da arquitetura MIPS. Usa-se a seguir as seguintes convenções:

Convenções Utilizadas na Tabela 1:

- ♦ **Rd (destination register)** é o registrador usado na maioria das instruções de três operandos como destino dos dados processados, especificado por um código binário de 5 bits. Veja a Seção 4 deste documento e para um detalhamento mais amplo refira-se ao Apêndice A do livro texto de Patterson & Hennessy;
- ♦ **Rs (source register)** e **Rt (target register)** são registradores usados em muitas instruções como origem e/ou destino das instruções. São especificados por códigos binários de 5 bits. Veja a Seção 4 deste documento para mais detalhes;
- ♦ Registradores específicos da máquina são indicados por **\$xx**, onde **xx** é o número decimal do registrador, entre 0 e 31;
- ♦ Os registradores **HI** e **LO** são especiais (ficam fora do banco de registradores de uso geral), sendo usados para armazenar resultados das instruções de multiplicação e divisão (**MULTU** e **DIVU**, respectivamente, na MIPS_S);
- ♦ O sinal **←** é usado para designar atribuição (escrita) de valores resultantes da avaliação da expressão à direita do sinal ao registrador ou à posição de memória identificada à esquerda do sinal;
- ♦ Os identificadores **imed16** e **imed26** representam operandos imediatos representados em 16 e 26 bits, respectivamente;
- ♦ O identificador **shamt** (do inglês, *shift amount*) representa a quantidade de bits a deslocar nas instruções **SLL**, **SLLV**, **SRA**, **SRAV**, **SRL** e **SRLV**. O operador **mod** produz o resto da divisão inteira do elemento à esquerda do **mod** pelo elemento à direita deste;
- ♦ O operador **&** representa a concatenação de vetores de bits;
- ♦ A expressão **PMEMD(X)** representa o conteúdo de uma posição de memória de dados cujo endereço é **X** (na leitura) ou a própria posição da memória de dados (na escrita). A quantidade de bits relevantes depende da instrução específica executada. Por exemplo, em **LW** são 32 bits, em **LBU** são 8 bits;
- ♦ Está implícito em todas as instruções o incremento do registrador **PC** após a busca da instrução. Na organização, isto se reflete no uso de um registrador temporário denominado **NPC**, que contém o valor do **PC** incrementado após a busca de uma instrução. Qualquer outra referência à manipulação do **PC** é parte da semântica da instrução particular;
- ♦ **Extensão de sinal** é a operação que transforma um dado vetor de bits em outro maior, mas cujo valor em complemento de 2 é equivalente ao valor representado pelo vetor menor. Consiste em copiar o bit de sinal do vetor, ou seja, o seu bit mais significativo, localizado na extrema esquerda deste vetor, tantas vezes quanto seja necessário, para gerar o vetor maior. Por exemplo, na instrução **ADDIU**, se **imed16** for **1111 1111 1111 1111** (-1 em complemento de 2, representado em 16 bits), a extensão de sinal transforma este vetor em **1111 1111 1111 1111 1111 1111 1111 1111** (-1 em complemento de 2, representado em 32 bits). O leitor deve perceber que esta operação é trivialmente correta para números positivos, onde o bit de sinal estendido é 0. Quando se menciona extensão de sinal, os valores imediatos são interpretados como números em complemento de 2. Caso contrário, os números são representações binárias puras, como ocorre na instrução **J⁴**;
- ♦ Operandos imediatos de instruções de salto são tratados distintamente de operandos de outras instruções (por exemplo, lógicas e aritméticas). Primeiro, as arquiteturas MIPS definem que o endereçamento de ambas as memórias, de Instrução e de Dados, é orientado a byte (ou seja, cada endereço nestas memórias é o endereço de um único byte). Segundo, as instruções sempre começam na Memória de Instruções em endereços que são múltiplos de 4 (ou seja, endereços onde os dois últimos bits são 00). Assim, seria um desperdício de armazenamento manter sempre esses 2 bits constantes em cada endereço e eles são, conseqüentemente, suprimidos no código de objeto de instruções. Isto implica que o processador deve manipular operandos de salto, adicionando estes 0s antes de usar o operando, o que é feito deslocando o operando 2 bits para a esquerda, enquanto se insere 0s à direita. Isso é verdadeiro para as instruções **BEQ**, **BGEZ**, **BLEZ**, **BNE**, **J** e **JAL**.

⁴ **CUIDADO!!!:** Existe um outro simulador bem conhecido, o SPIM, onde a extensão de sinal é feita sobre os 14 bits, e não sobre 16 bits (pode ser um *bug* do simulador ou uma definição da arquitetura; O importante é que isto afeta a implementação do processador aqui especificado).

Tabela 1 – Mnemônicos, codificação e semântica resumida de todas as instruções do processador MIPS_S. As constantes numéricas nos códigos de instrução são valores hexadecimais. Legenda: As operações lógicas são **and**, **or**, **xor** e **nor** (bit a bit); Rs, Rt e Rd são nomes de campos específicos da instrução que contêm os endereços de registradores, entre 0 e 31. O campo shamt representa uma quantidade de bits a deslocar de um registrador, entre 0 e 31. O campo fonte rótulo é transformado em um campo lmed16 ou lmed26, dependendo do modo de endereçamento do operando (relativo ou pseudo-absoluto). As operações aritméticas (+, -, *, /) operam apenas considerando operandos como números naturais (N). PMEMD representa o conteúdo da memória de dados no endereço dado pela expressão entre parênteses.

INSTRUÇÃO		FORMATO OBJETO DA INSTRUÇÃO						SEMÂNTICA DA AÇÃO DA INSTRUÇÃO
FORMATO FONTE DA INSTRUÇÃO		31 - 26	25 - 21	20 - 16	15 - 11	10 - 6	5 - 0	
ADDU	Rd, Rs, Rt	00	Rs	Rt	Rd	00	21	Rd ← Rs + Rt
SUBU	Rd, Rs, Rt	00	Rs	Rt	Rd	00	23	Rd ← Rs – Rt
AND	Rd, Rs, Rt	00	Rs	Rt	Rd	00	24	Rd ← Rs and Rt
OR	Rd, Rs, Rt	00	Rs	Rt	Rd	00	25	Rd ← Rs or Rt
XOR	Rd, Rs, Rt	00	Rs	Rt	Rd	00	26	Rd ← Rs xor Rt
NOR	Rd, Rs, Rt	00	Rs	Rt	Rd	00	27	Rd ← Rs nor Rt
SLL	Rd, Rt, shamt	00	00	Rt	Rd	shamt	00	Rd ← Rt deslocado shamt bits à esquerda (0s entram à direita)
SLLV	Rd, Rt, Rs	00	Rs	Rt	Rd	00	04	Rd ← Rt deslocado (Rs mod 32) bits à esquerda (0s entram à direita)
SRA	Rd, Rt, shamt	00	00	Rt	Rd	shamt	03	Rd ← Rt deslocado shamt bits à direita (mantendo o sinal)
SRAV	Rd, Rt, Rs	00	Rs	Rt	Rd	00	07	Rd ← Rt deslocado (Rs mod 32) bits à direita (mantendo o sinal)
SRL	Rd, Rt, shamt	00	00	Rt	Rd	shamt	02	Rd ← Rt deslocado shamt bits à direita (0s entram à esquerda)
SRLV	Rd, Rt, Rs	00	Rs	Rt	Rd	00	06	Rd ← Rt deslocado (Rs mod 32) bits à direita (0s entram à esquerda)
ADDIU	Rt, Rs, lmed16	09	Rs	Rt	lmed16			Rt ← Rs + (lmed16 com extensão de sinal)
ANDI	Rt, Rs, lmed16	0C	Rs	Rt	lmed16			Rt ← Rs and (lmed16 com extensão de 0)
ORI	Rt, Rs, lmed16	0D	Rs	Rt	lmed16			Rt ← Rs or (lmed16 com extensão de 0)
XORI	Rt, Rs, lmed16	0E	Rs	Rt	lmed16			Rt ← Rs xor (lmed16 com extensão de 0)
LUI	Rt, lmed16	0F	0	Rt	lmed16			Rt ← (lmed16 & 0x0000)
LBU	Rt, lmed16(Rs)	24	Rs	Rt	lmed16			Rt ← 0x000000 & PMEMD(lmed16 com extensão de sinal + Rs) (1 byte)
LW	Rt, lmed16(Rs)	23	Rs	Rt	lmed16			Rt ← PMEMD(lmed16 com extensão de sinal + Rs) (4 bytes)
SB	Rt, lmed16(Rs)	28	Rs	Rt	lmed16			PMEMD(lmed16 com extensão de sinal + Rs) ← Rt [7:0] (1 byte)
SW	Rt, lmed16(Rs)	2B	Rs	Rt	lmed16			PMEMD(lmed16 com extensão de sinal + Rs) ← Rt (4 bytes)
SLT	Rd, Rs, Rt	00	Rs	Rt	Rd	00	2A	Rd ← 1 se Rs < Rt (comparação como inteiros), senão Rd ← 0
SLTU	Rd, Rs, Rt	00	Rs	Rt	Rd	00	2B	Rd ← 1 se Rs < Rt (comparação como naturais), senão Rd ← 0
SLTI	Rt, Rs, lmed16	0A	Rs	Rt	lmed16			Rt ← 1 se Rs < lmed16 (comparação como inteiros), senão Rt ← 0
SLTIU	Rt, Rs, lmed16	0B	Rs	Rt	lmed16			Rt ← 1 se Rs < lmed16 (comparação como naturais), senão Rt ← 0
BEQ	Rs, Rt, rótulo	04	Rs	Rt	lmed16			PC ← NPC + (lmed16 * 4 com extensão de sinal), se Rs=Rt
BGEZ	Rs, rótulo	01	Rs	01	lmed16			PC ← NPC + (lmed16 * 4 com extensão de sinal), se Rs>=0
BLEZ	Rs, rótulo	06	Rs	00	lmed16			PC ← NPC + (lmed16 * 4 com extensão de sinal), se Rs<=0
BNE	Rs, Rt, rótulo	05	Rs	Rt	lmed16			PC ← NPC + (lmed16 * 4 com extensão de sinal), se Rs≠Rt
J	rótulo	02	lmed26					PC ← NPC[31:28] & lmed26 & “00”
JAL	rótulo	03	lmed26					\$31 ← NPC; PC ← NPC[31:28] & lmed26 & “00”
JALR	Rd, Rs	00	Rs	00	Rd	00	09	Rd ← NPC; PC ← Rs
JR	Rs	00	Rs	0000			08	PC ← Rs
MULTU	Rs, Rt	00	Rs	Rt	000		19	Hi & LO ← Rs * Rt
DIVU	Rs, Rt	00	Rs	Rt	000		1B	Lo ← Rs / Rt; Hi ← Rs mod Rt
MFHI	Rd	00	000		Rd	00	10	Rd ← Hi
MFLO	Rd	00	000		Rd	00	12	Rd ← Lo

3.1 Classes Funcionais de Instruções

A partir da Tabela 1, propõe-se a seguinte divisão das instruções em classes funcionais:

- As **instruções aritméticas** são **ADDU, SUBU, ADDIU, MULTU, DIVU, SRA, SRAV**;
- As **instruções lógicas** são **AND, OR, XOR, NOR, ANDI, ORI, XORI, SLL, SLLV, SRL e SRLV**;
- As **instruções de movimentação de dados** são **LUI, LBU, LW, SB, SW, MFHI e MFLO**;
- As **instruções de controle de fluxo de execução** são **BEQ, BGEZ, BLEZ, BNE, J, JAL, JALR e JR**;
- Existem também **instruções miscelâneas**, **SLT, SLTI, SLTU e SLTIU**.

3.2 Observações sobre a Semântica de Instruções no Processador MIPS_S

Algumas observações gerais e particulares sobre o conjunto de instruções são apresentadas a seguir.

- A arquitetura MIPS foi elaborada para privilegiar a simplicidade do conjunto de instruções sem, contudo, sacrificar sua flexibilidade. Devido à limitação de todas as instruções possuírem exatamente o mesmo tamanho, instruções em geral disponíveis em outros processadores estão ausentes no MIPS. Contudo, foi tomado cuidado no projeto desta arquitetura para que tal funcionalidade possa ser suprida de forma simples via o conceito de pseudo-instruções. **Pseudo-instruções** são instruções inexistentes em uma arquitetura, mas disponibilizadas ao programador em linguagem de montagem. Sua implementação mediante uso de uma sequência de instruções existentes é feita no código objeto pelo programa montador. Por exemplo, uma instrução capaz de carregar uma constante imediata de 32 bits em um registrador obviamente não existe, mas pode ser implementada por uma sequência de duas instruções, **LUI** e **ORI**. É possível a programas montadores para o processador MIPS_S disponibilizar uma pseudo-instrução **LI** com um operando imediato de 32 bits, gerando como código para esta uma sequência **LUI + ORI**. Por exemplo, existe no montador/simulador MARS uma pseudo-instrução (**LA**, do inglês *load address*) que usa **LUI + ORI** para implementar uma função bastante útil, carregar um registrador com o endereço de um rótulo (em inglês, *label*) do programa, seja este da região de dados ou da região de programas.
- As instruções **SLT, SLTU, SLTI e SLTIU** servem para suprir a ausência de sinais qualificadores de estado específicos na arquitetura MIPS_S. Elas escrevem no registrador destino da instrução ou a constante 0 ou a constante 1, para indicar o resultado de uma comparação de magnitude. Após executar uma destas instruções, pode-se testar o valor escrito neste registrador de destino contra a constante 0 (por exemplo, usando as instruções **BEQ, BGEZ, BNE** ou **BLEZ**, tendo como um dos operandos a constante 0, disponível no registrador **\$0** ou **\$zero**). Tais sequências de instruções permitem implementar todas as comparações, de igualdade ou magnitude (“menor”, “maior”, “menor ou igual” e “maior ou igual”).

Exercício: Proponha uma sequência de instruções do processador MIPS_S para implementar as seguintes pseudo-instruções: **BGT, BGE, BLT e BLE** (respectivamente, **salta se maior, se maior ou igual, se menor e se menor ou igual**). Note-se ainda que as variações das instruções **SLTxx** permitem executar comparações de números naturais (as com sufixo **U**, do inglês *unsigned*) e de números inteiros representados em complemento de 2 (as sem sufixo **U**).

4 BANCO DE REGISTRADORES DO PROCESSADOR MIPS_S

A partir desta Seção, apresenta-se uma organização para o processador MIPS_S. Algumas das características mencionadas aqui provêm da arquitetura do MIPS. Isto será explicitado no texto a seguir, quando for o caso. O leitor pode desde já referir-se ao diagrama de blocos da organização proposta, apresentado na Figura 3 da Seção 5, de forma a acompanhar a leitura a seguir com uma ilustração adequada das estruturas mencionadas e de sua interconexão a demais estruturas presentes no hardware.

A organização contém o seguinte conjunto de registradores:

- Um banco de registradores contendo 32 registradores de uso geral, cada um de 32 bits, denominados **\$0** a **\$31**. **Esta é uma característica definida na arquitetura MIPS**. Existe uma denominação textual alternativa para cada um dos registradores. Ela está apresentada na Tabela 2 abaixo, retirada do Apêndice A do livro **Organização e Projeto de Computadores de D. A. Patterson & J. L. Hennessy**. Este Apêndice é de domínio público e pode ser facilmente encontrado na Internet⁵. O banco de registradores proposto aqui tem uma porta de

⁵ Existe uma versão do Apêndice A, anotada e corrigida em vários pontos onde o texto original apresenta incorreções ou omissões. Este material é disponibilizado.

escrita e duas portas de leitura. Isto significa que é possível escrever em apenas um registrador por vez, porém é possível fazer, em paralelo com a escrita mencionada, duas leituras simultâneas, colocando o conteúdo de um registrador no barramento de saída R1 e o conteúdo de outro registrador (ou do mesmo registrador) no barramento de saída R2;

- Registradores **R1**, **R2** e **R3**: São registradores que irão conter os operandos da maioria das instruções executadas;
- **RALU**: um registrador de 32 bits usado para armazenar o resultado produzido na ALU;
- **HI** e **LO**: são dois registradores de 32 bits definidos na arquitetura MIPS usados para armazenar o resultado da execução das instruções **MULTU** e **DIVU**. Para manipular estes resultados, existem as instruções **MFHI** e **MFLO**, que movem valores de **HI** ou **LO** para um registrador de uso geral. A necessidade destes registradores deriva do fato de uma multiplicação de dois números de 32 bits gerar, no pior caso, um valor de 64 bits, bem como do fato de uma divisão (inteira) gerar dois resultados de 32 bits, o quociente e o resto. No caso da instrução **DIVU**, o quociente é colocado em **LO** e o resto é colocado em **HI**;
- **MDR** (*memory data register*): é registrador que recebe dados provenientes da memória ou de outro local que produza um valor que necessite ser escrito no banco de registradores no último ciclo de uma instrução. Ele também possui 32 bits;

Tabela 2 – Denominações dos registradores do banco de registradores da arquitetura MIPS_S. Aconselha-se que os registradores a serem usados como registradores de trabalho em programas sejam os registradores 2 a 25 (\$v0 a \$t9). Os demais são reservados para operações de controle do montador, sistema operacional, simulação de pilhas etc. O montador MARS aceita a denominação da primeira ou da terceira colunas da tabela, sempre iniciadas com \$.

Nome	Número (hexa/binário)	Nome Alternativo	Significado ou Convenção de Utilização
\$0	00 / 00000	\$zero	constante 0
\$1	01 / 00001	\$at	reservado para o programa montador
\$2	02 / 00010	\$v0	resultado de função
\$3	03 / 00011	\$v1	resultado de função
\$4	04 / 00100	\$a0	argumento para função
\$5	05 / 00101	\$a1	argumento para função
\$6	06 / 00110	\$a2	argumento para função
\$7	07 / 00111	\$a3	argumento para função
\$8	08 / 01000	\$t0	temporário
\$9	09 / 01001	\$t1	temporário
\$10	0A / 01010	\$t2	temporário
\$11	0B / 01011	\$t3	temporário
\$12	0C / 01100	\$t4	temporário
\$13	0D / 01101	\$t5	temporário
\$14	0E / 01110	\$t6	temporário
\$15	0F / 01111	\$t7	temporário
\$16	10 / 10000	\$s0	temporário (salvo nas chamadas de função/subrotina)
\$17	11 / 10001	\$s1	temporário (salvo nas chamadas de função/subrotina)
\$18	12 / 10010	\$s2	temporário (salvo nas chamadas de função/subrotina)
\$19	13 / 10011	\$s3	temporário (salvo nas chamadas de função/subrotina)
\$20	14 / 10100	\$s4	temporário (salvo nas chamadas de função/subrotina)
\$21	15 / 10101	\$s5	temporário (salvo nas chamadas de função/subrotina)
\$22	16 / 10110	\$s6	temporário (salvo nas chamadas de função/subrotina)
\$23	17 / 10111	\$s7	temporário (salvo nas chamadas de função/subrotina)
\$24	18 / 11000	\$t8	temporário
\$25	19 / 11001	\$t9	temporário
\$26	1A / 11010	\$k0	reservado para o SO
\$27	1B / 11011	\$k1	reservado para o SO
\$28	1C / 11100	\$gp	apontador de área global
\$29	1D / 11101	\$sp	stack pointer
\$30	1E / 11110	\$fp	frame pointer
\$31	1F / 11111	\$ra	armazena endereço de retorno de subrotinas

Normalmente, sempre há a necessidade de existirem registradores adicionais, dependendo da organização implementada. Na organização proposta aqui, o valor do registrador **PC** é atualizado após a busca, mas o novo valor não é imediatamente escrito no **PC**, mas em um registrador auxiliar **NPC**, sigla derivada da terminologia *new program counter*, em inglês. Apenas no último ciclo de relógio de uma instrução o valor de **NPC** (ou outro, dependendo da instrução) é escrito no **PC**. Os valores lidos do banco de registradores são armazenados nos registradores **R1** e **R2**, e o valor obtido pela execução de uma dada operação lógico-aritmética é armazenado no registrador **RALU**. Apresentam-se detalhes nas Seções a seguir.

5 UMA ORGANIZAÇÃO PARA O PROCESSADOR MIPS_S

Na organização proposta aqui para o processador MIPS_S, a execução de qualquer instrução requer 4 a 5 ciclos de relógio, com a exceção das instruções **MULTU** e **DIVU**. Cada um dos 4/5 ciclos executa um conjunto limitado de partes de uma instrução e são assim denominados:

- **Ciclo 1: busca da instrução** - Comum a todas as instruções.

O BC/BD empregam um conjunto de **12** sinais de controle, organizados em **6** classes:

- [illegible]

O BD gera seis (6) sinais qualificadores, quatro (4) dos quais são enviados para o BC, para que este tome algumas decisões (sinais *end_mul*, *end_div*, *inst_Bgroup* e *jump*). Os demais são usados apenas internamente no BD (sinais *inst_Rgroup* e *inst_lgroup*).

O sinal de controle do multiplexador **M1** deriva dos sinais **i**, **inst_Bgroup** e **jump**. Os sinais de controle dos multiplexadores **M2** a **M11** são gerados no BC, mas derivam diretamente da instrução corrente identificada por **i/uins.i**. Para uma ideia mais completa da implementação do BD do processador MIPS_S, resta apresentar: (i) a organização interna do banco de registradores; (ii) a organização da unidade lógica e aritmética (ALU); (iii) a estrutura das operações de multiplicação e divisão; e (iv) o Bloco de Controle (*control_unit* ou BC) do MIPS_S. As três Seções a seguir discutem as organizações dos três primeiros blocos citados, enquanto a Seção 7 discute o BC.

5.1 Banco de Registradores

A Figura 4 ilustra a organização do banco de registradores, sob a forma de um diagrama de blocos. A organização do banco de registradores inclui os **32** registradores em si e a implementação das duas portas de leitura e da porta de escrita, bem como da decodificação do endereço de escrita para geração da habilitação de escrita do registrador em questão. As portas de leitura consistem em multiplexadores (32 X 32):(1 X 32), controladas pelos endereços de leitura. A porta de escrita consiste em um decodificador 5 → 32 com sinal de habilitação do processo de decodificação. O processo de decodificação consiste em, a partir do identificador do endereço do registrador a escrever (**AdWP**, de 5 bits) gerar os 32 sinais de habilitação de escrita, apenas 1 dos quais pode estar ativado (em **'1'**) em um dado ciclo de relógio. Deve estar claro que, se **ce='0'**, nenhum dos sinais de habilitação de escrita dos registradores estará em **'1'**, pois isto indica que não deve haver escrita de registrador no ciclo de relógio corrente.

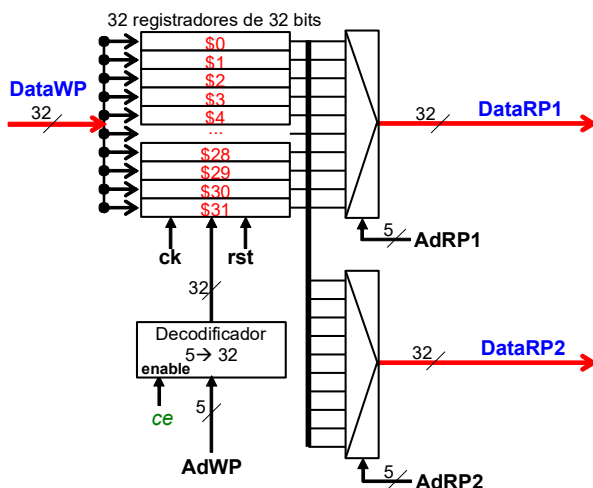


Figura 4 - Diagrama em blocos do banco de registradores de uso geral da organização MIPS_S.

5.2 Resumo das Operações Executadas pela ALU

A Tabela 3 define a operação básica do hardware da ALU. Esta última é um hardware puramente combinacional que possui duas entradas de dados e uma saída de dados. Além disto, a ALU possui uma entrada de controle para informar a operação a ser realizada a cada instante, derivada da instrução em execução no momento.

Tabela 3 – Operações da ALU no processador MIPS_S, para cada instrução. As operações lógicas (**and**, **or**, **xor**, **nor**) operam bit a bit. As operações **x << y** e **x >> y** correspondem ao deslocamento de Y bits de X à direita e à esquerda respectivamente.

Instruções	Operação da ALU
ADDU, ADDIU, LBU, LW, SB, SW, BEQ, BGEZ, BLEZ, BNE	OP2 + OP1
SUBU	OP2 - OP1
AND, ANDI	OP2 and OP1
OR, ORI	OP2 or OP1
XOR, XORI	OP2 xor OP1
NOR	OP2 nor OP1
SLL	OP1 << OP2[10:6] c/ 0s entrando à direita
SLLV	OP2 << OP1[4:0] c/ 0s entrando à direita
SRA	OP1 >> OP2[10:6] c/ cópia(s) do bit 31 entrando à esquerda
SRAV	OP2 >> OP1[4:0] c/ cópia(s) do bit 31 entrando à esquerda
SRL	OP1 >> OP2[10:6] c/ 0s entrando à esquerda
SRLV	OP2 >> OP1[4:0] c/ 0s entrando à esquerda
LUI	OP2[15:0] & 0x"0000"
SLT, SLTI	1 se OP1 < OP2 (comparação com sinal), senão 0
SLTU, SLTIU	1 se OP1 < OP2 (comparação sem sinal), senão 0
J, JAL	OP1[31:28] & OP2[27:0]

JR, JALR	OP1
MULTU, DIVU, MFHI, MFLO	Nenhuma operação

A operação da ALU é relevante para quase todas as instruções, embora algumas destas apenas a utilizem para transportar um valor da entrada para a saída (como JR e JALR) e outras (como MULTU e DIVU) sequer utilizem a ALU. O resultado da operação na ALU é sempre armazenado no registrador RALU. A Tabela 3 define quais devem ser as operações da ALU, para cada instrução. A ALU possui uma outra saída

5.3 Organização para as instruções MULTU e DIVU

As instruções MULTU e DIVU, que realizam a **multiplicação e a divisão de números naturais**, devido a sua intrínseca maior complexidade, são implementadas fora da ALU. Aqui, as instruções DIVU e MULTU são implementadas de maneira estrutural, usando texto VHDL sintetizável. Para tanto, escolheu-se usar algoritmo de implementação de um multiplicador e um divisor seriais em hardware, tal como descrito nas transparências 10 a 17 da apresentação disponível no link mostrado abaixo.

<http://www.inf.pucrs.br/~calazans/undergrad/arq1/aulas/aritcomp.pdf>.

O algoritmo de multiplicação é aquele descrito nas transparências acima. O algoritmo de divisão é o algoritmo sem restauração. Estes e outros algoritmos fáceis de implementar sob a forma de um pequeno bloco de dados e uma máquina de estados de controle estão descritos em algum detalhe no livro **Organização e Projeto de Computadores de D. A. Patterson & J. L. Hennessy**, na Seção 4.6 da segunda edição deste livro.

6 CICLOS PARA EXECUTAR INSTRUÇÕES DA ORGANIZAÇÃO MIPS_S

Dada a descrição da organização do bloco de dados da MIPS_S apresentada na Seção anterior, é possível sumarizar o tempo de execução de todas as instruções em termos de número de ciclos de relógio, o que é mostrado na Tabela 4 abaixo. Note-se que estes são valores específicos para esta organização.

Tabela 4 – Número de ciclos gastos para buscar e executar instruções na organização do processador MIPS_S.

INSTRUÇÃO	NUMERO DE CICLOS	INSTRUÇÃO	NUMERO DE CICLOS
ADDU	4	SB	4
SUBU	4	SW	4
AND	4	SLT	4
OR	4	SLTU	4
XOR	4	SLTI	4
NOR	4	SLTIU	4
SLL	4	BEQ	4
SLLV	4	BGEZ	4
SRA	4	BLEZ	4
SRAV	4	BNE	4
SRL	4	J	4
SRLV	4	JAL	4
ADDIU	4	JALR	4
ANDI	4	JR	4
ORI	4	MULTU	67 ⁶
XORI	4	DIVU	67 ⁷
LUI	4	MFHI	4
LBU	5	MFLO	4
LW	5		

7 O Bloco de Controle (BC) do processador MIPS_S

Para comandar a execução de instruções no MIPS_S, o BC do MIPS_S possui um conjunto de quatro estruturas de hardware. São estas:

1. Um **registrador** que guarda o código da instrução atualmente em execução (IR);
2. Uma estrutura para controlar a posição de memória onde reside a instrução atual e a próxima a ser executada, composta por **dois registradores (PC e NPC)** e um **incrementador**;

⁶ Valor específico, depende da organização escolhida para o multiplicador.

⁷ Valor específico, depende da organização escolhida para o divisor.

3. **A decodificação de instruções**, uma tabela em hardware que, para cada código de 32 bits contido no **IR**, identifica a instrução que este representa, ou o fato de o código não se referir a nenhuma instrução válida no MIPS_S (valor **invalid_instruction**);
4. **Uma máquina de estados** que gera a sequência de sinais de controle usados em cada ciclo de execução de uma dada instrução.

A Figura 5 ilustra a estrutura da máquina de estados de controle da MIPS_S, onde o próximo estado é função: (i) do estado atual; (ii) da instrução armazenada no registrador **IR**; (iii) de alguns sinais de controle usados em processos específicos (além obviamente dos sinais de controle **clock** e **reset**, a transição de estados depende dos sinais **suspend**, **end_mul** e **end_div**).

- **IR (instruction register)**: armazena o código de operação (*opcode*) da instrução atual e, na maioria dos casos, código(s) de algum(ns) operando(s) desta. É um registrador de 32 bits.
- **PC (program counter)**: é o contador de programa. É um registrador de 32 bits.

A Figura 5 também ilustra quais registradores são alterados, através da indicação de qual/quais sinal/sinais de controle de habilitação de escrita é/são ativado/ativados em cada estado. Não se mostra as ativações dos sinais de controle dos multiplexadores, ou as operações da ALU. Segue abaixo a lista de nome e função de cada um dos 7 estados da máquina:

- **Sfetch**: primeiro ciclo, estado inicial da máquina, realiza a busca de instrução, computa e armazena o incremento do PC;
- **SReg**: segundo ciclo, leitura dos registradores fonte para a instrução corrente e busca de operandos (no BD);
- **Salu**: terceiro ciclo, operação com a ALU e eventual uso do comparador, ou do hardware de multiplicação ou do hardware de divisão;
- **Swbk**: quarto ciclo para a maioria das instruções, onde se escreve o resultado no banco de registradores e atualiza-se o contador de programa (é o quinto ciclo das instruções **LW** e **LBU**);
- **Sld**: quarto ciclo das instruções **LW** e **LBU**, onde se lê um dado da Memória de Dados;
- **Sst**: último ciclo (quarto) das instruções **SW** e **SB**, onde se escreve um dado na Memória de Dados;
- **Sjump**: último ciclo (quarto) das instruções (quarto) de salto condicional ou incondicional, atualiza-se o valor do PC. Condicionalmente, se escreve algo no banco de registradores (para as instruções **JAL** e **JALR**).

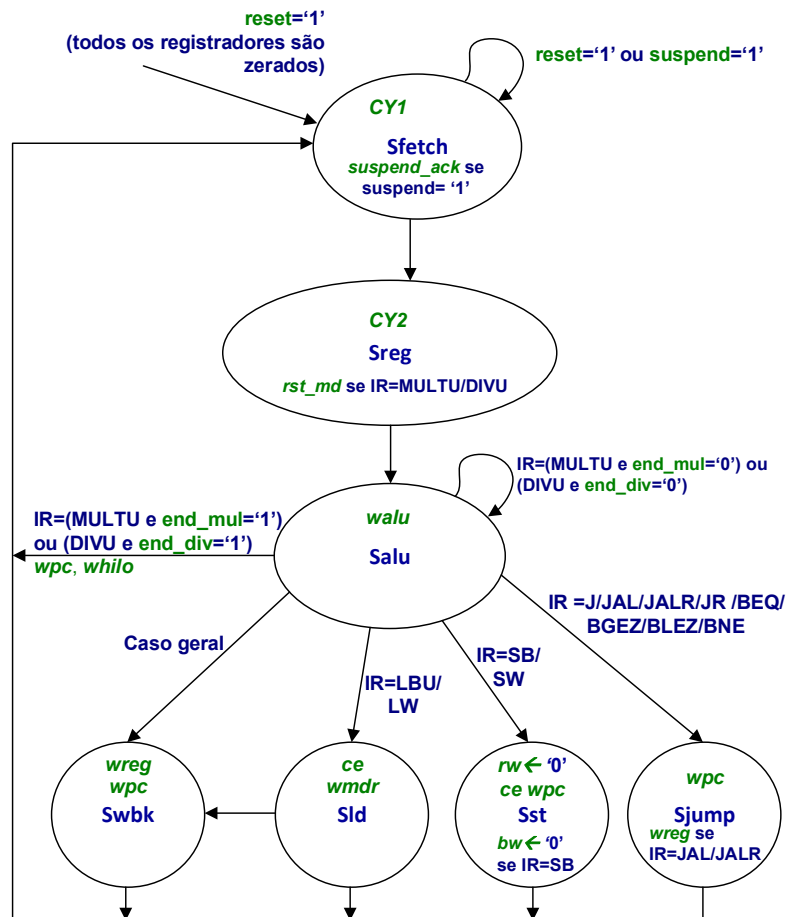


Figura 5 - Máquina de estados de controle para a organização MIPS_S.

8 PROGRAMA PARA TESTAR TODAS AS INSTRUÇÕES DA MIPS_S

A Figura 6 mostra a interface gráfica do simulador MARS, composta de 5 subjanelas (4 subjanelas estão ocultas, aquela da aba Edit, que mostra o editor de textos do ambiente MARS, duas outras para uso de coprocessadores, **Coproc 1** e **Coproc 2** e uma para operações de entrada e saída, **RUN I/O**):

- a subjanela **Text Segment** mostra as instruções do programa ora em simulação, em três formatos: código-objeto (**Code**), código intermediário (**Basic**) e código fonte (**Source**);
- a subjanela **Data Segment** mostra os dados estáticos do programa ora em simulação (várias outras áreas de memória estão disponíveis para visualização, tais como a área de pilha apontada pelo registrador **\$sp**, o *heap* etc.);
- a subjanela **Labels** mostra dados sobre os rótulos do programa, tanto da área de instruções como da área de dados;
- a subjanela **Registers** mostra os conteúdos dos registradores da arquitetura (os 32 registradores do banco, o **PC**, o **HI** e o **LO**);
- a subjanela **Mars Messages** mostra mensagens que o montador/simulador gera para o usuário durante a carga, montagem e execução de programas.

A ferramenta de montagem tem como entrada o nome do programa descrito em linguagem de montagem (tipicamente com nome **<file>.asm**). É possível gerar um arquivo de saída com o conteúdo das janelas de instruções e dados do programa, incluindo o código objeto gerado. Este mais tarde é usado, após algum tratamento, como entrada para a simulação do hardware do processador.

O código objeto pode ser gerado após a leitura de um arquivo contendo um programa corretamente descrito e a montagem do código objeto a partir deste.

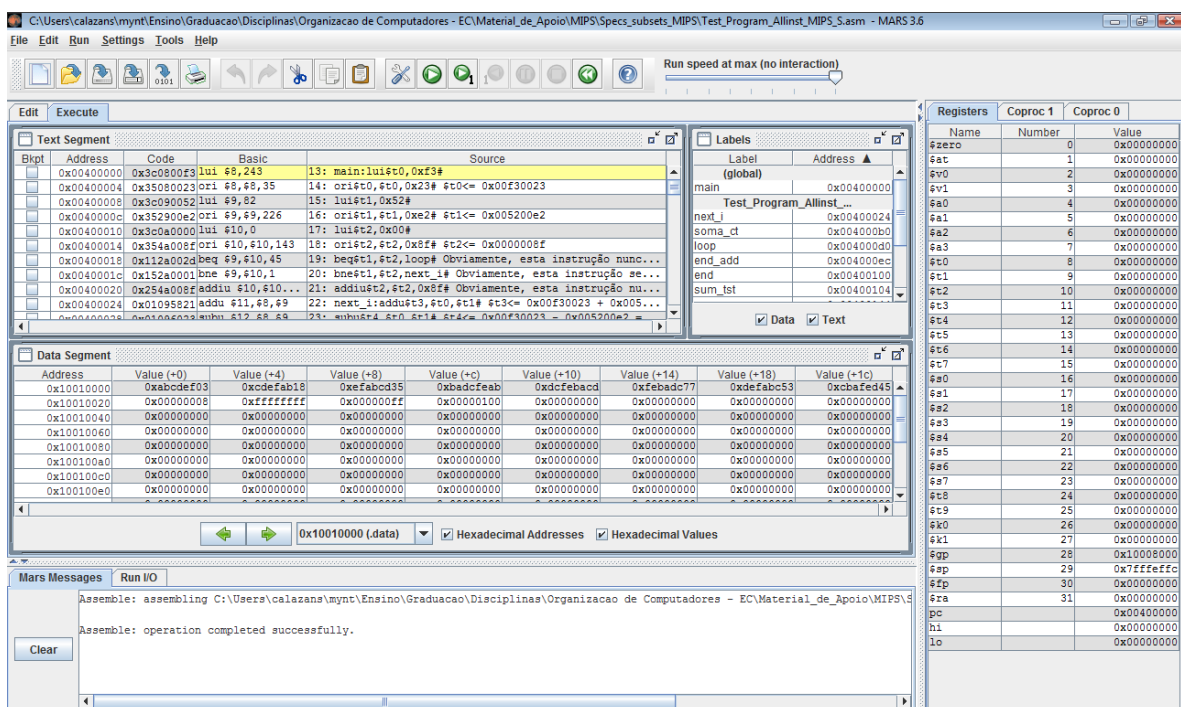


Figura 6 - Interface gráfica do montador/simulador MARS.

Recomenda-se **escrever os programas em linguagem de montagem (assembly)**, gerando-se o código objeto automaticamente, a partir do montador/simulador MARS. Ferramentas de simulação, assim como a documentação de como utilizar a ferramenta encontram-se disponíveis, e são encontradas facilmente na Internet.

A Figura 7 apresenta um programa escrito em linguagem de montagem para o processador MIPS_S, e que executa pelo menos uma instância de cada uma das 37 instruções deste processador. Recomenda-se o uso do simulador MARS para executar este programa e verificar sua funcionalidade, cujos efeitos são descritos linha a linha no programa, no campo de comentários.

O simulador MARS pode gerar arquivos de saída contendo o código das áreas de dados e de instruções de qualquer programa, entre outras informações. Tais arquivos podem ser salvos usando a opção de **menu File → Dump Memory** do simulador. Eles são úteis para gerar a entrada do processo de simulação da implementação de hardware.

```
#####
## ARQUITETURA MIPS_S_Sim - teste de todas as instrucoes
## Autor: Ney Calazans (nlvcalazans@gmail.com)
## Teste eh exaustivo quanto ao total de instrucoes, nao quanto ha
## cobertura de caso de testes possiveis
##
## Para simular este programa no Hw do MIPS_S_Sim,
## simule por 13,76 microssegundos (assumindo um clock de 50Mhz)
## simule por 15,86 microssegundos, caso TB use o teste de suspend
## Ultima alteracao: 06/05/2025
#####
.test          # Declaracao de inicio do segmento de texto
.globl main    # Declaracao de que o rotulo main e global
#####
# testes de instrucoes individuais
#####
main:          #
    lui        $t0,0xf3          # $t0<= 0x00f30023
    ori        $t0,$t0,0x23      # $t0<= 0x00f30023
    lui        $t1,0x52          # $t1<= 0x005200e2
    ori        $t1,$t1,0xe2      # $t1<= 0x005200e2
    lui        $t2,0x00          # $t2<= 0x0000008f
    ori        $t2,$t2,0x8f      # $t2<= 0x0000008f
    beq        $t1,$t2,loop      # Obviamente, esta instrucao nunca deve saltar
    bne        $t1,$t2,next_i    # Obviamente, esta instrucao sempre deve saltar
    addiu      $t2,$t2,0x8f      # Obviamente, esta instrucao nunca deve executar
next_i:        #
    addu       $t3,$t0,$t1       # $t3<= 0x00f30023 + 0x005200e2 = 0x01450105
    subu       $t4,$t0,$t1       # $t4<= 0x00f30023 - 0x005200e2 = 0x00a0ff41
    subu       $t5,$t1,$t1       # $t5<= 0x0
    and        $t6,$t0,$t1       # $t6<= 0x00f30023 and 0x005200e2 = 0x00520022
    or        $t7,$t0,$t1       # $t7<= 0x00f30023 or 0x005200e2 = 0x00f300e3
    xor        $t8,$t0,$t1       # $t8<= 0x00f30023 xor 0x005200e2 = 0x00a100c1
    nor        $t9,$t0,$t1       # $t9<= 0x00f30023 nor 0x005200e2 = 0xff0cfff1c
    multu      $t0,$t1          # Hi & Lo <= 0x00f30023 * 0x005200e2 = 0x00004dd6e1bc1ee6
    mfhi       $s0              # $s0<= 0x00004dd6
    mflo       $s1              # $s1<= 0xe1bc1ee6
    divu       $t0,$t1          # Hi,Lo<= 0x00f30023 (mod), 0x005200e2 (/) = 0x4efe5f,0x00000002
    addiu      $t0,$t0,0x00ab     # $t0<= 0x00f30023 + 0x000000ab = 0x00f300ce
    andi       $t0,$t0,0x00ab     # $t0<= 0x00f300ce and 0x000000ab = 0x0000008a
    xori       $t0,$t0,0xffab     # $t0<= 0x0000008a xor 0x0000ffab = 0x0000ff21
    sll        $t0,$t0,4          # $t0<= 0x000ff210 (deslocado 4 bits para a esquerda)
    srl        $t0,$t0,9          # $t0<= 0x000007f9 (deslocado 9 bits para a direita)
    addiu      $s2,$zero,8        # $s2<= 0x00000008
    sllv       $t0,$t0,$s2        # $t0<= 0x0007f900 - desloca $t0 8 bits a esquerda, inserindo 0s
    sllv       $t0,$t0,$s2        # $t0<= 0x07f90000 - desloca $t0 8 bits a esquerda, inserindo 0s
    sllv       $t0,$t0,$s2        # $t0<= 0xf9000000 - desloca $t0 8 bits a esquerda, inserindo 0s
    sra        $t0,$t0,4          # $t0<= 0xff900000 - desloca $t0 4 bits a direita mantendo o sinal
    srav       $t0,$t0,$s2        # $t0<= 0xffff9000 - desloca $t0 8 bits a direita mantendo o sinal
    srlv       $t0,$t0,$s2        # $t0<= 0x00ffff90 - desloca $t0 8 bits a direita, inserindo 0s
    la         $t0,array          # coloca em $t0 o endereco inicial do vetor array (0x10010000)
    lbu        $t1,$($t0)         # $t1<= 0x0000000e (primeiro byte terceiro byte do segundo elemento)
    xori       $t1,$t1,0xff       # $t1<= 0x00000010, inverte byte inferior
    sb         $t1,6($t0)         # CUIDADO, mudou elemento do array a ser processado por soma_ct
    addiu      $t0,$zero,0x1       # $t0<= 0x00000001
    subu       $t0,$zero,$t0       # $t0<= 0xffffffff
    bgez       $t0,loop           # Esta instrucao nunca deve saltar, pois $t0 = -1
    slt        $t3,$t0,$t1        # $t3<= 0x00000001, pois -1 < 10
    sltu       $t3,$t0,$t1        # $t3<= 0x00000000, pois (2^32)-1 > 10
    slti       $t3,$t0,0x1        # $t3<= 0x00000001, pois -1 < 1
    sltiu      $t3,$t0,0x1        # $t3<= 0x00000000, pois (2^32)-1 > 1
#####
# soma uma constante a um vetor
#####
soma_ct:       #
    la         $t0,array          # coloca em $t0 o endereco do vetor (0x10010000)
    la         $t1,$t0.size       # coloca em $t1 o endereco do tamanho do vetor 0x10010020
    lw         $t1,0($t1)         # coloca em $t1 o tamanho do vetor 0x8
    la         $t2,const          # coloca em $t2 o endereco da constante 0x10010024
    lw         $t2,0($t2)         # coloca em $t2 a constante, 0xffffffff (-1)
loop:          #
    blez       $t1,end_add        # se/quando tamanho torna-se 0, fim do processamento
    lw         $t3,0($t0)         # coloca em $t3 o proximo elemento do vetor
    addu       $t3,$t3,$t2        # soma constante
    sw         $t3,0($t0)         # atualiza no vetor o valor do elemento
    addu       $t0,$t0,4          # atualiza ponteiro do vetor. Lembrar, 1 palavra=4 posicoes de memoria
    addu       $t1,$t1,-1         # decrementa contador de tamanho do vetor
    j          loop              # continua execucao
#####
# teste de subrotinas aninhadas
#####
end_add:       #
    li         $sp,0x10010800     # Para poder simular o hardware, inicializa-se o $sp c/valor adequado
    addiu      $sp,$sp,-4         # Assume-se uma RAM de 2Kbytes e aponta-se para a primeira posicao depois do fim dela
    sw         $ra,0($sp)         # assume-se $sp inicializado, aloca espaco na pilha
    jal        sum_tst            # no MARS o $sp inicializado com 0x7FFFEFFC. Usamos 0x10010800.
    lw         $ra,0($sp)         # salva endereco de retorno de quem chamou (trap handler) 0x00000000
    addiu      $sp,$sp,4          # salta para subrotina sum_tst
    jr         $ra               # ao retornar, recupera endereco de retorno da pilha
    # atualiza apontador de pilha
    # FIM DO PROGRAMA AQUI. Volta para o "sistema operacional" 0x00000000
# Inicio da primeira subrotina: sum_tst
sum_tst:       #
    la         $t0,var_a          # pega endereco da primeira variavel (pseudo-instrucao) 0x10010028
    lw         $t0,0($t0)         # toma o valor de var_a e coloca em $t0 0x000000FF
    la         $t1,var_b          # pega endereco da segunda variavel (pseudo-instrucao) 0x1001002C
    lw         $t1,0($t1)         # toma o valor de var_b e coloca em $t1 0x00000100
    addu       $t2,$t1,$t0        # soma var_a com var_b e coloca resultado em $t2 0x000001FF
    addiu      $sp,$sp,-8         # aloca espaco na pilha, $sp vai para 0x100107F4
    sw         $t2,0($sp)         # no topo da pilha coloca o resultado da soma 0x000001FF
    sw         $ra,4($sp)         # abaixo do topo coloca o endereco de retorno 0x00400100
    la         $t3,ver_ev         # pega endereco da subrotina ver_ev (pseudo-instrucao) 0x0040014C
    jalr       $ra,$t3            # chama subrotina que verifica se resultado da soma eh par em 0x0040014C
    lw         $ra,4($sp)         # ao retornar, recupera endereco de retorno da pilha 0x00400100
    addiu      $sp,$sp,8          # atualiza apontador de pilha para 0x100107FC
    jr         $ra               # Rotina acaba AQUI. Retorna para quem chamou, em 0x00400100
# Inicio da segunda subrotina: ver_ev. Trata-se de subrotina folha, que nao usa pilha
ver_ev:        #
    lw         $t3,0($sp)         # tira dados do topo da pilha (parametro) 0x000001FF, e coloca em $t3
    andi       $t3,$t3,1         # $t3 <= 1 se parametro eh impar, 0 caso contrario, resulta em 0x00000001
    jr         $ra               # e retorna para 0x00400140
#####
# Area de Dados
#####
# para trecho que soma constante a vetor
array:         #
    .word      0xABCD0F03 0xCDEFAB18 0xEFABCD35 0xBADCFEAB 0xDCFEABCD 0xFEBADC77 0xDEFABC53 0xCBFAFED45
    # terceiro byte da segunda palavra (0xEF) vira 0x10 antes de executar soma_ct
size:          #
    .word      0x8
const:         #
    .word      0xffffffff         # constante -1 em complemento de 2
    # dados para trecho de teste de chamadas de subrotinas

```

Figura 7 - Programa exemplo para teste de todas as instruções da organização MIPS_S.